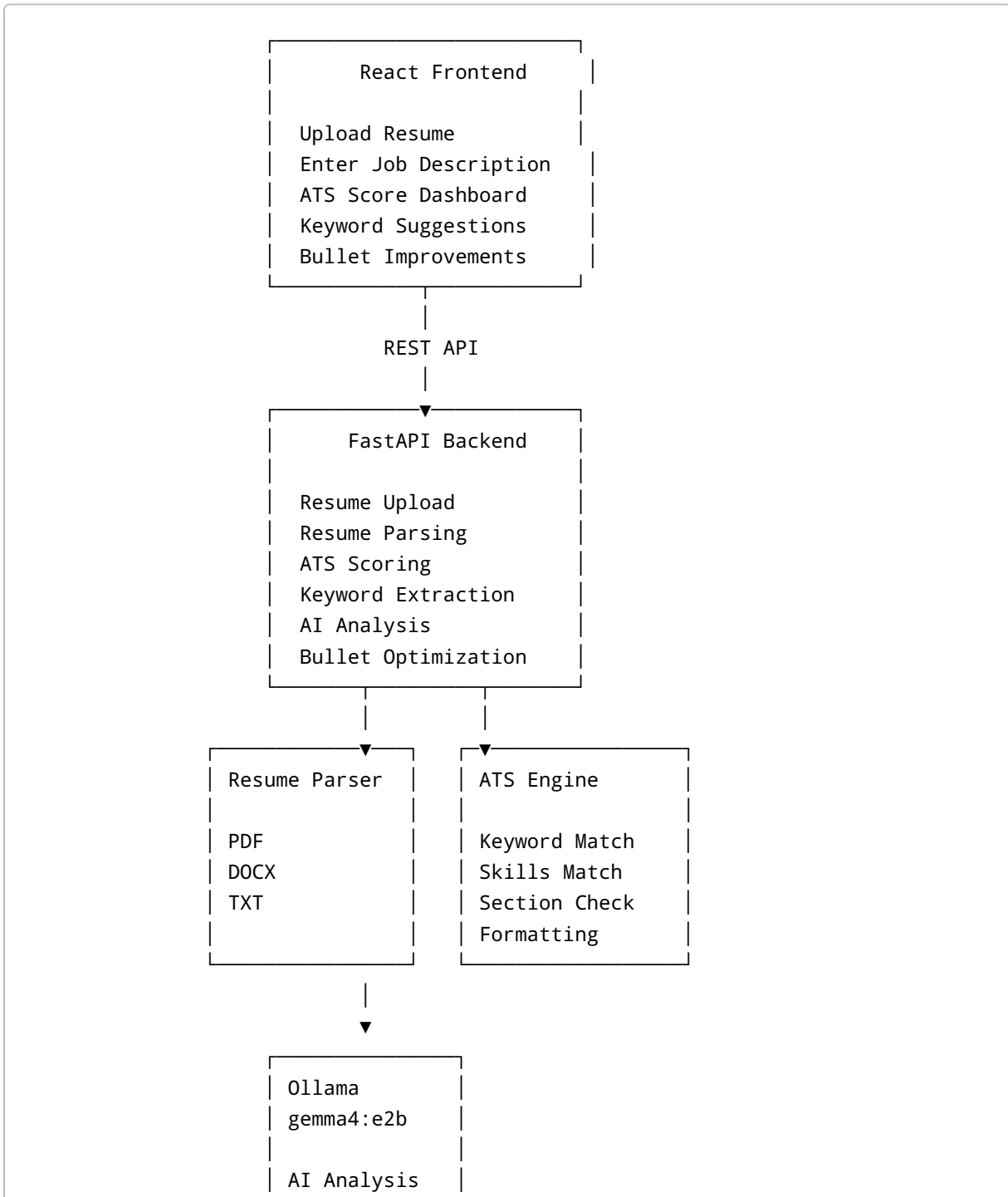


AI Resume Analyzer & ATS Optimizer

1. Recommended High-Level Architecture

- [] Build the application using a **frontend + backend + local AI** architecture.



2. Technology Stack

Backend

- [] **Python 3.12+**
- [] **FastAPI** for REST APIs
- [] **Pydantic** for request/response validation
- [] **Uvicorn** as the ASGI server
- [] **httpx** for communicating with Ollama
- [] **PyMuPDF (fitz)** for PDF extraction
- [] **python-docx** for DOCX extraction
- [] **NLTK / spaCy** for NLP processing
- [] **scikit-learn** for similarity calculations
- [] **RapidFuzz** for fuzzy keyword matching
- [] **SQLAlchemy** if persistent storage is required

Recommended backend structure

```
backend/  
├── app/  
│   ├── main.py  
│   ├── api/  
│   │   ├── resume.py  
│   │   ├── analysis.py  
│   │   ├── jobs.py  
│   │   └── health.py  
│   ├── core/  
│   │   ├── config.py  
│   │   ├── logging.py  
│   │   └── security.py  
│   ├── models/  
│   │   ├── resume.py  
│   │   ├── job.py  
│   │   └── analysis.py  
│   └── schemas/
```

```
|
|
|   ├── resume.py
|   ├── analysis.py
|   └── suggestions.py
|
|   ├── services/
|   |   ├── resume_parser.py
|   |   ├── ats_engine.py
|   |   ├── keyword_engine.py
|   |   ├── similarity_engine.py
|   |   ├── ollama_service.py
|   |   └── bullet_optimizer.py
|   |
|   ├── prompts/
|   |   ├── resume_analysis.txt
|   |   ├── keyword_analysis.txt
|   |   └── bullet_optimization.txt
|   |
|   └── utils/
|       ├── text_cleaner.py
|       └── file_utils.py
|
|── tests/
|── requirements.txt
|── README.md
```

3. Frontend

Recommended Stack

- **React**
- **TypeScript**
- **Vite**
- **Tailwind CSS**
- **shadcn/ui** or another component library
- **Axios** or native `fetch`
- **Recharts** for score visualization

Frontend structure

```
frontend/
├── src/
│   ├── components/
│   │   └── ResumeUploader.tsx
```

```
├── JobDescriptionInput.tsx
├── ATSScore.tsx
├── KeywordMatch.tsx
├── MissingKeywords.tsx
├── ResumeIssues.tsx
├── BulletOptimizer.tsx
├── ScoreBreakdown.tsx
├── pages/
│   ├── Home.tsx
│   ├── Analysis.tsx
│   └── History.tsx
├── services/
│   └── api.ts
├── types/
│   └── analysis.ts
├── App.tsx
├── package.json
└── README.md
```

Support these formats initially:

- [\[\] PDF](#)
- [\[\] DOCX](#)
- [\[\] TXT](#)

Do **not** start with every possible format.

Validate:

- [] File extension
- [] MIME type
- [] Maximum file size
- [] Number of pages
- [] Empty/corrupted documents
- [] Password-protected PDFs
- [] Malicious filenames
- [] Duplicate uploads

Recommended flow:

```
Upload
↓
Validate
↓
Extract text
↓
Normalize text
↓
Detect sections
↓
Create structured Resume JSON
```

5. Resume Parser

The parser should convert an unstructured resume into structured data.

Example:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "+91XXXXXXXXXX",
  "summary": "...",
  "skills": [
    "Python",
    "AWS",
    "Docker",
    "Kubernetes"
  ],
  "experience": [
    {
      "company": "ABC Technologies",
      "role": "DevOps Engineer",
      "duration": "2022-2025",
      "bullets": [
        "Managed Kubernetes clusters...",
        "Automated CI/CD pipelines..."
      ]
    }
  ],
  "education": [],
```

```
"certifications": [],  
"projects": []  
}
```

Implement section detection for:

- [] Summary
- [] Objective
- [] Experience
- [] Education
- [] Skills
- [] Projects
- [] Certifications
- [] Achievements
- [] Publications
- [] Languages

Use deterministic rules first and LLM assistance only when necessary.

6. ATS Scoring Engine

This should be one of the most important architectural decisions.

Do not ask the LLM to directly generate the ATS score.

Instead:

```
Resume  
+  
Job Description  
↓  
Deterministic ATS Engine  
↓  
Score Components  
↓  
Weighted Final Score  
↓  
LLM Explanation
```

For example:

```
ATS Score =
```

Keyword Match	30%
Skills Match	20%
Job Title Match	10%
Experience Relevance	15%
Resume Structure	10%
Formatting	10%
Achievements	5%

Make these weights configurable rather than hardcoding them permanently.

7. Keyword Matching

Extract keywords from the job description.

Categorize them into:

Technical Skills
Soft Skills
Tools
Frameworks
Cloud Platforms
Certifications
Job Titles
Domain Terms
Methodologies

Example JD:

Looking for a DevOps Engineer with AWS, Kubernetes, Docker, Terraform, Jenkins, Python and CI/CD experience.

Extract:

AWS
Kubernetes
Docker
Terraform
Jenkins
Python

CI/CD
DevOps Engineer

Then classify:

Matched:

- ✓ AWS
- ✓ Kubernetes
- ✓ Docker
- ✓ Python

Missing:

- x Terraform
- x Jenkins
- x CI/CD

8. Semantic Keyword Matching

Exact matching alone is insufficient.

For example:

Job Description:
Amazon Web Services

Resume:
AWS

These should match.

Similarly:

Continuous Integration / Continuous Deployment
CI/CD

should match.

Implement a keyword normalization layer:

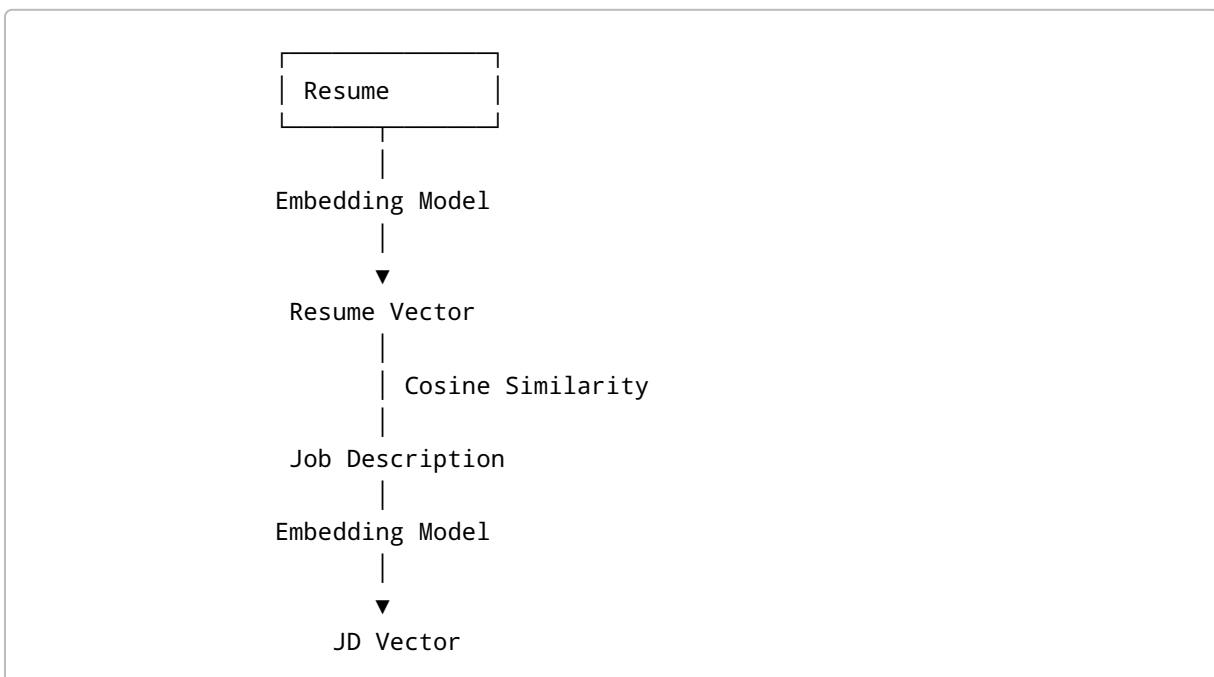
AWS → Amazon Web Services
K8s → Kubernetes
CI/CD → Continuous Integration / Continuous Deployment
Postgres → PostgreSQL
JS → JavaScript
TS → TypeScript

Then optionally use embeddings for semantic similarity.

9. Embedding Layer

For a stronger version of the project, add an embedding model separately from Gemma.

Architecture:



You can initially skip this and use:

- TF-IDF
- keyword matching
- fuzzy matching

Then add embeddings in Version 2.

This keeps Version 1 simple.

10. Ollama Integration

Your current setup:

```
Ollama
└─ gemma4:e2b
```

should be treated as an external local AI service.

Do not tightly couple your business logic directly to Ollama.

Create:

```
ollama_service.py
```

with methods such as:

```
analyze_resume()
extract_keywords()
analyze_job_description()
improve_bullet()
generate_summary()
```

The rest of your application should simply call:

```
AIService
```

rather than knowing whether the implementation is Ollama, OpenAI, or another model.

This gives you the ability to swap models later.

11. AI Responsibilities

Use Gemma for tasks where language understanding is useful.

Good LLM tasks

- [] Analyze resume quality

- ☐ Identify weak bullet points
- ☐ Suggest stronger wording
- ☐ Identify missing contextual skills
- ☐ Explain why a keyword matters
- ☐ Generate improved bullet points
- ☐ Generate professional summaries
- ☐ Compare resume experience against JD
- ☐ Detect vague statements
- ☐ Suggest measurable achievements

Avoid using LLM for

- ☐ File validation
- ☐ Page counting
- ☐ Exact keyword counting
- ☐ Email extraction
- ☐ Phone extraction
- ☐ ATS score calculation
- ☐ Duplicate detection
- ☐ Basic section detection

Those should remain deterministic.

12. Bullet Point Optimizer

This can become one of the strongest features.

Input:

Managed servers and deployments.

AI output:

Automated application deployments across Linux environments,
reducing manual deployment effort and improving release consistency.

But enforce rules.

The AI should:

- ☐ Never invent experience
- ☐ Never invent metrics
- ☐ Never invent technologies

- [] Preserve factual meaning
- [] Use strong action verbs
- [] Incorporate relevant JD terminology when truthful
- [] Keep bullets concise
- [] Prefer Action + Task + Result structure

Example prompt constraint:

Do not introduce technologies, metrics, responsibilities, certifications, employers, or achievements that are not supported by the provided resume.

This is extremely important for a resume application.

13. Prompt Architecture

Do not put giant prompts directly inside Python code.

Use:

```
prompts/  
├─ resume_analysis.txt  
├─ keyword_extraction.txt  
├─ bullet_optimizer.txt  
├─ job_match.txt  
└─ summary_optimizer.txt
```

Use structured output wherever possible.

For example:

```
{  
  "issues": [],  
  "strengths": [],  
  "missing_keywords": [],  
  "matched_keywords": [],  
  "recommendations": []  
}
```

Then validate the LLM response with Pydantic.

14. ATS Formatting Checks

Build deterministic checks for:

- ☐ Standard section names
- ☐ Excessive tables
- ☐ Text embedded inside images
- ☐ Missing contact information
- ☐ Unusual symbols
- ☐ Excessive columns
- ☐ Headers/footers
- ☐ Very small fonts
- ☐ Excessive whitespace
- ☐ Inconsistent date formats
- ☐ Inconsistent job titles
- ☐ Extremely long paragraphs
- ☐ Missing bullet points
- ☐ Excessive decorative elements

Give each issue a severity:

CRITICAL
WARNING
INFO

15. Score Breakdown

Do not show only:

ATS Score: 78

Show:

ATS Score: 78/100

Keyword Match	82%
Skills Match	90%
Experience	76%
Formatting	95%

Structure	88%
Achievements	61%

Then explain:

Why your score is 78

- + Strong AWS and Kubernetes alignment
- + Good technical skills coverage
- + Standard resume structure
- Terraform missing
- Several bullets lack measurable outcomes
- Professional summary doesn't target the role

16. Job Description Input

The application should support:

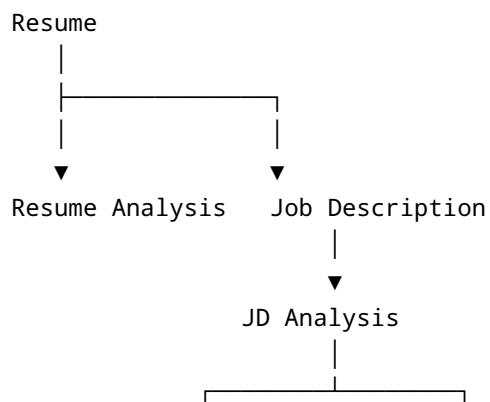
Resume only

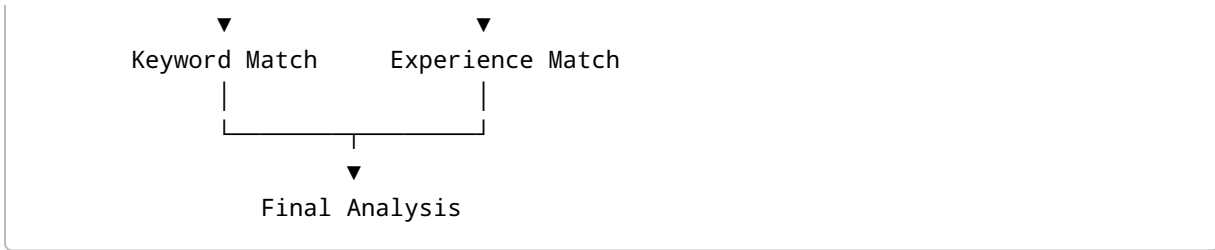
and:

Resume + Job Description

The second mode should provide significantly deeper analysis.

Flow:





17. Database

For Version 1, you can actually avoid a database.

Use:

SQLite

if you want local analysis history.

Store:

```
analysis_id
resume_hash
created_at
resume_metadata
job_description
ats_score
analysis_result
```

Avoid storing the original resume permanently unless necessary.

For a future multi-user version:

PostgreSQL

would be the better choice.

18. Privacy Architecture

Because resumes contain personal information, make privacy a core feature.

- ☐ Keep AI processing local
- ☐ Never send resume content to external APIs
- ☐ Do not commit uploaded resumes to Git
- ☐ Add `.gitignore` rules
- ☐ Store temporary uploads outside source directories
- ☐ Delete temporary files after processing
- ☐ Avoid logging resume contents
- ☐ Avoid logging email addresses and phone numbers
- ☐ Add a "Delete analysis" feature

Recommended:

```
uploads/  
temp/  
data/
```

should all be excluded from Git.

19. Git Repository Structure

Recommended repository:

```
ai-resume-ats-optimizer/  
|  
├─ backend/  
├─ frontend/  
├─ tests/  
├─ docs/  
|   ├─ architecture.md  
|   ├─ ats-scoring.md  
|   └─ api.md  
|  
├─ .github/  
|   └─ workflows/  
|       └─ ci.yml  
|  
├─ .gitignore  
└─ .env.example
```



```
|— docker-compose.yml
|— README.md
|— LICENSE
└— Makefile
```

20. GitHub Security

Never commit:

```
.env
resume files
personal documents
database files
logs
model files
Ollama model binaries
```

Your repository should contain:

```
.env.example
```

instead of:

```
.env
```

Example:

```
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=gemma4:e2b

DATABASE_URL=sqlite:///./resume_analyzer.db

MAX_UPLOAD_SIZE_MB=10
```

21. API Design

Recommended endpoints:

```
GET /api/health

POST /api/resume/upload

POST /api/analyze

POST /api/analyze/keywords

POST /api/analyze/bullet

POST /api/analyze/summary

GET /api/analysis/{id}

DELETE /api/analysis/{id}
```

A combined endpoint could accept:

```
{
  "resume_id": "abc123",
  "job_description": "..."
```

and return:

```
{
  "ats_score": 82,
  "score_breakdown": {},
  "matched_keywords": [],
  "missing_keywords": [],
  "formatting_issues": [],
  "experience_analysis": {},
  "bullet_suggestions": []
}
```

22. Testing Strategy

Create tests for each layer.

Unit tests

- ☐ PDF parsing
- ☐ DOCX parsing
- ☐ Section detection
- ☐ Keyword extraction
- ☐ Keyword normalization
- ☐ Fuzzy matching
- ☐ ATS calculation
- ☐ Formatting detection

AI tests

Create a small collection of fixed resumes and job descriptions.

Test:

Input → Expected characteristics

rather than requiring exact LLM wording.

For example:

Expected:

- Terraform identified as missing
- AWS identified as matched
- Bullet contains action verb
- No fabricated technology

23. CI/CD

Since the project will be on GitHub:

- ☐ GitHub Actions
- ☐ Python linting
- ☐ Python formatting
- ☐ Unit tests
- ☐ TypeScript build
- ☐ Frontend linting
- ☐ Security checks

Recommended tools:

```
Python:
Ruff
Pytest
MyPy

Frontend:
ESLint
Prettier
TypeScript
```

Do not make CI dependent on your local Gemma model initially.

Test the Ollama integration separately.

24. Docker

You can containerize:

```
Frontend
Backend
Database
```

But I would **not initially put Ollama inside your application container.**

Instead:

```
Docker
├─ frontend
├─ backend
└─ postgres (optional)

Host
└─ Ollama
    └─ gemma4:e2b
```

Your backend communicates with:

```
http://localhost:11434
```

or the appropriate host address when running inside Docker.

This makes your local AI dependency easier to manage.

25. Recommended Development Phases

Phase 1 — MVP

- ☐ React UI
- ☐ FastAPI backend
- ☐ PDF upload
- ☐ DOCX upload
- ☐ Resume text extraction
- ☐ Job description input
- ☐ Basic keyword extraction
- ☐ Exact/fuzzy keyword matching
- ☐ Basic ATS score
- ☐ Ollama integration
- ☐ AI bullet optimization

Target:

```
Upload Resume
  ↓
Paste JD
  ↓
Analyze
  ↓
ATS Score
  ↓
Matched/Missing Keywords
  ↓
AI Recommendations
```

Phase 2 — Better ATS Engine

- ☐ Resume section detection
- ☐ Formatting checks
- ☐ Skill taxonomy
- ☐ Synonym mapping
- ☐ Semantic similarity
- ☐ Better score weighting
- ☐ Detailed score explanations
- ☐ Resume vs JD comparison

Phase 3 — Advanced AI

- ☐ Embedding model
 - ☐ Semantic skill matching
 - ☐ Experience relevance scoring
 - ☐ AI-generated resume summary
 - ☐ Bullet-by-bullet optimization
 - ☐ Job-specific resume recommendations
 - ☐ Multiple resume versions
 - ☐ Analysis history
-

Phase 4 — Production

- ☐ Authentication
 - ☐ PostgreSQL
 - ☐ Background job processing
 - ☐ Redis
 - ☐ Rate limiting
 - ☐ Audit logging
 - ☐ File cleanup
 - ☐ Docker deployment
 - ☐ CI/CD
 - ☐ Monitoring
-

26. Recommended Final Architecture

For your first serious implementation, I recommend:

Frontend

React + TypeScript
Vite
Tailwind CSS
shadcn/ui
Recharts



Backend

FastAPI
Pydantic
SQLAlchemy
PyMuPDF
python-docx
spaCy
scikit-learn
RapidFuzz



ATS Engine

Keyword matching
Skill matching
Formatting
Section detection
Similarity



Storage

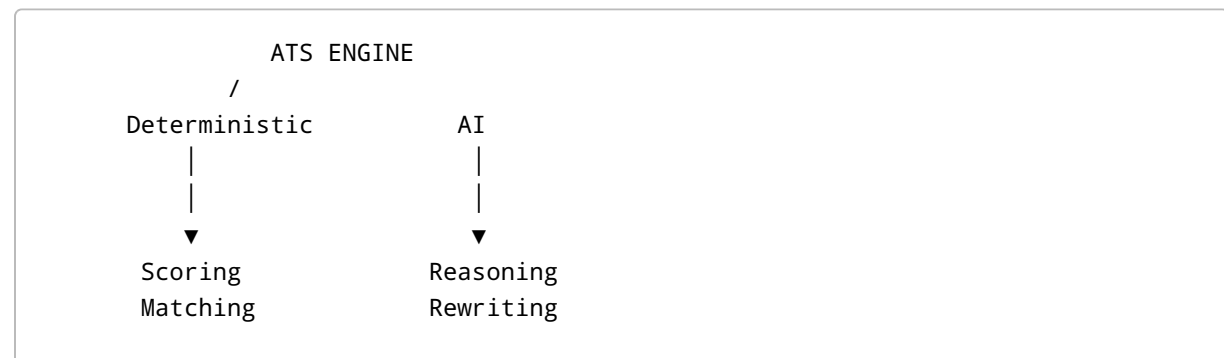
SQLite initially
PostgreSQL later

AI Service

Ollama
gemma4:e2b
Structured output
Prompt templates
Pydantic validation

27. One Important Design Principle

Keep this boundary extremely clear:



Parsing
Validation

Suggestions
Explanation

The AI should enhance the ATS engine, not replace it.

This will make your project much more reliable and also give you a stronger GitHub portfolio project because you can demonstrate that the system combines **traditional NLP + deterministic scoring + local LLM inference** rather than simply sending the entire resume to an LLM and asking it for a score.

28. Suggested GitHub Project Positioning

A good project description would be:

AI-powered, privacy-first resume analyzer and ATS optimizer using FastAPI, React, NLP, and locally hosted Ollama LLMs. The system analyzes resumes against job descriptions, calculates a transparent ATS compatibility score, identifies missing keywords, detects formatting issues, and uses a local LLM to generate factual, job-specific resume improvements.

That gives you a technically interesting portfolio project covering:

```
Full-stack development
+
NLP
+
LLM integration
+
Prompt engineering
+
Information extraction
+
Similarity algorithms
+
AI evaluation
+
Docker
+
CI/CD
+
Privacy-first architecture
```

For your current machine, I would start with `gemma4:e2b` and avoid adding a second generative model. Build the deterministic ATS engine first, then add semantic embeddings as the next major enhancement.